



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ  
УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА

Институт радиоэлектроники и информационных технологий  
Кафедра информатики и систем управления

Драйвер для программирования кнопок мыши макросами

ОТЧЕТ

по курсовой работе

по дисциплине

Проектирование системного программного обеспечения

РУКОВОДИТЕЛЬ:

\_\_\_\_\_

(подпись)

Ляхманов Д.А.

СТУДЕНТ:

\_\_\_\_\_

(подпись)

Браилко В.А.

С22-СИБ

(шифр группы)

Работа защищена «\_\_» \_\_\_\_\_

С оценкой \_\_\_\_\_

# Введение

В условиях активного развития современных операционных систем с открытым исходным кодом особую значимость приобретает системное программирование как область, обеспечивающая эффективное и гибкое взаимодействие программного обеспечения с аппаратными средствами. Особое место в данной области занимает разработка драйверов устройств, функционирующих на уровне ядра операционной системы и определяющих базовое поведение периферийных устройств ввода. В данной работе рассматривается операционная система Linux – одна из наиболее распространённых платформ с открытым исходным кодом, предоставляющая развитую архитектуру для разработки драйверов уровня ядра. Одним из наиболее распространённых устройств ввода является компьютерная мышь, от корректной и гибкой работы которой зависит удобство и эффективность пользовательского взаимодействия.

Актуальность работы заключается в том, что современные игровые и профессиональные мыши оснащены дополнительными программируемыми кнопками, однако в операционной системе Linux отсутствует универсальное встроенное средство для настройки этих кнопок на уровне ядра. Существующие пользовательские решения (например, `xbindkeys`, `pipер`, `libratbag`) работают в пространстве пользователя и зависят от графической среды, демонов или прав доступа. Такие подходы не обеспечивают универсальности: они не функционируют в текстовом режиме, в играх с прямыми вызовами к устройствам ввода, а также могут вносить задержки из-за переключения контекста между пространствами.

Разработка драйвера уровня ядра для программирования макросов на кнопках мыши представляет собой комплексную практическую задачу, требующую глубокого понимания архитектуры операционной системы Linux. В ходе решения подобных задач изучаются ключевые концепции системного программирования, включая разграничение пространства ядра и пользовательского пространства, работу подсистемы ввода, а также принципы безопасной и устойчивой разработки программного кода, функционирующего с повышенными привилегиями. Это придаёт работе значительную образовательную и исследовательскую ценность.

Программирование макросов на кнопках мыши имеет практическую применимость в ряде сценариев. Подобная функциональность может использоваться в игровой индустрии для быстрого выполнения сложных комбинаций, в профессиональных средах разработки и редактирования (CAD, графические редакторы) для автоматизации рутинных действий, таких как копирование (Ctrl+C) и вставка (Ctrl+V). Реализация обработки событий мыши на уровне драйвера ядра позволяет добиться прозрачности для

пользовательских приложений, минимальных задержек и независимости от конкретной графической оболочки или программного окружения.

Таким образом, разработка драйвера программирования макросов на кнопках мыши является актуальной задачей, находящейся на пересечении образовательных, исследовательских и практических аспектов системного программирования.

Целью курсовой работы является разработка драйвера операционной системы Linux, реализующего программирование макросов на дополнительных кнопках компьютерной мыши на уровне ядра. Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить архитектуру подсистемы ввода операционной системы Linux.
2. Рассмотреть принципы работы HID-устройств и драйверов мыши.
3. Изучить механизмы взаимодействия драйверов с ядром и способы обработки событий мыши.
4. Спроектировать архитектуру драйвера, реализующего программирование макросов на кнопках мыши и демонстрирующего применение методов системного программирования на уровне ядра.
5. Создать соответствующее программное решение с поддержкой назначения макросов через интерфейс sysfs.
6. Обеспечить корректную эмуляцию сочетаний клавиш (например, Ctrl+C, Ctrl+V) и поддержку задержки выполнения макроса.
7. Провести тестирование разработанного драйвера и проанализировать результаты его работы.

# **Глава 1. Теоретические основы**

## ***1.1. Понятие драйверов устройств и их роль в операционной системе***

Драйвер устройства – это программный компонент операционной системы, обеспечивающий взаимодействие между аппаратным обеспечением и пользовательскими приложениями. Основная задача драйвера заключается в абстрагировании особенностей конкретного устройства и предоставлении стандартного интерфейса для работы с ним. В операционных системах семейства Linux драйверы являются неотъемлемой частью ядра. Они позволяют операционной системе управлять различными аппаратными устройствами, такими как клавиатуры, мыши, дисковые накопители, сетевые адаптеры и другие периферийные устройства. Без соответствующего драйвера устройство либо не будет распознано системой, либо не сможет корректно функционировать.

На сегодняшний день Linux – монолитное ядро с поддержкой загружаемых модулей. Драйверы могут быть встроены непосредственно в ядро или загружаться в виде модулей. Драйверы устройств и расширения ядра обычно запускаются в нулевом кольце защиты, с полным доступом к оборудованию. В отличие от обычных монолитных ядер, драйверы устройств легко собираются в виде модулей и загружаются или выгружаются во время работы системы. Использование модулей ядра позволяет добавлять или удалять поддержку устройств без перезагрузки системы, что существенно упрощает разработку, отладку и сопровождение драйверов.

## ***1.2. Архитектура ядра Linux и подсистема ввода***

Ядро Linux включает в себя множество подсистем, каждая из которых отвечает за определённый аспект функционирования операционной системы. Одной из таких подсистем является подсистема ввода (Input Subsystem), предназначенная для обработки событий от устройств ввода. Подсистема ввода обеспечивает единый механизм работы с такими устройствами, как:

1. клавиатуры;
2. мыши;
3. тачпады;
4. джойстики;
5. сенсорные экраны.

Основная идея данной подсистемы заключается в разделении логики работы устройства и обработки пользовательских событий. Устройства ввода регистрируются в системе, после чего их события передаются в пользовательское пространство через стандартные интерфейсы. Подсистема

ввода управляет устройствами ввода через специальные драйверы, работая через `/dev/input/`, где каждый файл `eventX` представляет устройство; драйверы преобразуют «сырые» события (нажатие клавиши, движение мыши) в унифицированный формат, понятный приложениям.

### ***1.3. HID-устройства и стандарт Human Interface Device***

Human Interface Device (HID) – это стандарт, предназначенный для описания и унификации устройств взаимодействия пользователя с компьютерной системой. К HID-устройствам относятся клавиатуры, мыши, джойстики, графические планшеты, сенсорные панели и другие устройства ввода, передающие данные о действиях пользователя. Основной целью стандарта HID является обеспечение универсального способа обмена данными между устройствами ввода и операционной системой, независимо от конкретного производителя и аппаратной реализации. Благодаря этому устройства, соответствующие стандарту HID, могут использовать типовые драйверы операционной системы без необходимости установки специализированного программного обеспечения.

В операционной системе Linux поддержка HID-устройств реализована в виде отдельной подсистемы ядра. Центральным компонентом данной подсистемы является HID core, отвечающий за обработку HID-отчётов, разбор HID-дескрипторов и взаимодействие с подсистемой ввода. Взаимодействие между HID-устройством и ядром Linux осуществляется с помощью отчётов (HID reports), которые могут быть одного из трёх типов: Input, Output и Feature. Для устройств ввода, таких как компьютерная мышь, используются в основном отчёты типа Input, содержащие информацию о перемещении и состоянии кнопок.

Каждый HID-отчёт формируется в соответствии с HID-дескриптором – структурой данных, описывающей формат передаваемой информации и назначение элементов управления устройства. Ядро Linux анализирует HID-дескриптор и сопоставляет полученные данные с логическими элементами ввода, после чего преобразует их в унифицированные события подсистемы ввода. Если устройство правильно реализует протокол своего класса, то для него не нужно разрабатывать специальный драйвер и система автоматически определит его назначение и начнёт им управлять. В случае, если по каким-то причинам нужно изменить поведение устройства, необходимо написать собственный HID-драйвер.

### ***1.4. Подсистема evdev и обработка событий мыши***

Для передачи событий ввода в пользовательское пространство в Linux используется интерфейс `evdev` (event device). Для каждого зарегистрированного устройства ввода ядро создаёт символьное устройство в каталоге `/dev/input/` с

именем eventX, где X – уникальный номер устройства. Через данные символьные устройства пользовательские приложения могут получать события ввода путём чтения соответствующих файлов. Такой подход позволяет абстрагироваться от аппаратных особенностей конкретного устройства и использовать единый интерфейс взаимодействия с устройствами ввода.

События evdev передаются в виде структур типа struct input\_event, определённых в заголовочном файле linux/input.h. Каждое событие содержит следующие основные поля:

1. тип события (type) – определяет категорию события;
2. код события (code) – уточняет конкретное действие;
3. значение (value) – содержит числовое значение события.

Для компьютерной мыши в подсистеме evdev используются следующие типы событий:

1. EV\_REL – относительное перемещение устройства;
2. REL\_X – перемещение по оси X;
3. REL\_Y – перемещение по оси Y;
4. REL\_WHEEL – прокрутка колеса мыши;
5. EV\_KEY – нажатие и отпускание кнопок мыши.

Перемещение мыши интерпретируется как относительное смещение, то есть каждое событие содержит величину изменения координаты, а не абсолютное положение курсора. Для обеспечения корректной обработки нескольких связанных событий используется событие типа EV\_SYN. Оно сигнализирует о завершении передачи группы событий и позволяет приложениям интерпретировать их как единое логическое действие. Например, при перемещении мыши одновременно по осям X и Y формируются события REL\_X и REL\_Y, после чего передаётся событие синхронизации EV\_SYN, указывающее на завершение данного набора данных.

Пользовательские приложения и графические среды получают события ввода, считывая данные из файлов /dev/input/eventX. Чтение может осуществляться напрямую либо с использованием специализированных библиотек, которые упрощают работу с событиями evdev и обеспечивают дополнительную обработку данных. На основании полученных событий графическая подсистема выполняет соответствующие действия.

## **Глава 2. Проектирование драйвера программирования макросов на кнопках мыши**

### **2.1. Постановка задачи**

Целью разработки является создание драйвера для мыши, реализующего функцию программирования макросов на дополнительных кнопках (например, боковых кнопках DPI, Forward/Back). Драйвер должен обеспечивать возможность:

1. назначения произвольной последовательности клавиш на каждую из дополнительных кнопок мыши;
2. поддержки сочетаний клавиш (например, LeftCtrl + C);
3. настройки макросов через стандартный интерфейс sysfs;
4. (опционально) выполнения макроса с задержкой (0.5 секунды);
5. сохранения совместимости с существующими приложениями и подсистемой ввода Linux.

В рамках поставленной задачи драйвер должен работать с конкретными HID-устройствами – мышами Logitech G502, Razer DeathAdder и другими, с возможностью расширения списка поддерживаемых устройств.

### **2.2. Анализ возможных архитектурных решений и выбор способа реализации**

Для решения задачи программирования макросов в операционной системе Linux существует несколько архитектурных подходов, отличающихся уровнем вмешательства в систему, переносимостью, сложностью реализации и областью применимости. Рассмотрим основные из них.

#### **1. Реализация на уровне оконной системы (X11/Wayland)\*\***

Наиболее простой способ – использование утилит вроде xbindkeys или xmodmap, которые перехватывают события ввода на уровне X11 и запускают скрипты или эмулируют клавиши. Однако данный подход имеет существенный недостаток: он работает только в рамках графической сессии и не влияет на события ввода в консоли, играх, встроенных системах или приложениях, напрямую обращающихся к устройствам ввода через /dev/input/eventX. Следовательно, он не обеспечивает универсальности, требуемой для системного решения.

#### **2. Реализация в пользовательском пространстве с использованием uinput\*\***

Более гибкое решение – создание демона в userspace, который:

- получает события от физической мыши через интерфейс evdev (/dev/input/eventX);

- при обнаружении нажатия дополнительной кнопки эмулирует виртуальное устройство ввода с помощью механизма `input`;
- отправляет последовательность клавиш.

Такой подход обеспечивает высокую переносимость, простоту отладки, безопасность (падение демона не ведёт к краху системы). Однако вносит небольшую задержку из-за переключения между пространствами, требует управления правами доступа к устройствам ввода и дополнительных мер по отключению оригинального устройства или фильтрации событий.

### 3. Реализация через `input_handler`

Данный метод перехватывает уже распарсенные `input`-события (`EV_KEY`, `BTN_EXTRA`) и перенаправляет их в виртуальное `input`-устройство с модифицированными значениями. Создаёт дополнительное устройство ввода, требует аккуратного управления жизненным циклом (выделение/освобождение памяти, регистрация `handle`), возможен риск заикливания.

### 4. Реализация HID-драйвера ядра

Драйвер перехватывает уже распарсенные HID-события в обработчике `event` (или сырые отчёты в `raw_event`), определяет код нажатой кнопки и эмулирует последовательность клавиш через виртуальное устройство ввода. Работает на уровне ядра, независимо от графической подсистемы или пользовательской сессии; обеспечивает минимальную задержку и полную замену поведения устройства для всей системы.

Несмотря на существование нескольких рабочих решений, для основной реализации был выбран метод HID-драйвера. Он соответствует учебной цели курсовой работы, а также данный подход обеспечит выполнение всех требований: программирование макросов на системном уровне, сохранение совместимости с приложениями и возможность масштабирования на другие устройства. Обеспечивает полную гибкость, контроль над логикой преобразования.

## **2.3. Проектирование алгоритма работы драйвера**

Ниже представлена блок-схема алгоритма работы драйвера программирования макросов на кнопках мыши.

инициализация → ожидание события → проверка кода кнопки → если макрос назначен → запуск `work` с задержкой → выполнение макроса → эмуляция клавиш через `input_dev`

Каждое входящее событие от мыши проверяется на соответствие известным кодам дополнительных кнопок (`BTN_EXTRA`, `BTN_SIDE` и др.). В случае совпадения и наличия назначенного макроса запускается отложенная работа

(delayed\_work) с таймером 500 мс. По истечении времени выполняется эмуляция нажатия клавиш через виртуальное устройство ввода. Такой подход обеспечивает прозрачную обработку всех входящих событий и корректную работу драйвера на системном уровне.

## Глава 3. Реализация драйвера

### 3.1. Описание среды разработки

Разработка и тестирование драйвера выполнялись на физической машине под управлением операционной системы Debian GNU/Linux 13 (trixie) с ядром Linux версии 6.12.57+deb13-amd64.

Драйвер реализован в виде загружаемого модуля ядра Linux, что позволяет подключать и отключать его без перезагрузки системы. Исходный код драйвера написан на языке C, так как данный язык является основным для разработки компонентов ядра Linux.

Для компиляции использовались:

1. компилятор gcc, входящий в стандартный набор GNU;
2. заголовочные файлы текущей версии ядра, расположенные в каталоге `/lib/modules/$(uname -r)/build`;
3. утилита make, обеспечивающая автоматизацию процесса сборки модуля;
4. система сборки ядра Linux Kbuild.

Сборка модуля выполнялась с использованием стандартного Kbuild-совместимого Makefile.

В процессе разработки, загрузки и отладки драйвера использовались стандартные утилиты операционной системы Linux, предназначенные для работы с модулями ядра и устройствами ввода.

1. Для управления жизненным циклом модуля ядра применялись следующие команды:

- `insmod` – используется для загрузки скомпилированного модуля (.ko) непосредственно в ядро Linux. Данная утилита применялась при тестировании драйвера после каждой сборки;

- `rmmod` – используется для выгрузки модуля из ядра, что позволяет корректно завершать работу драйвера и повторно загружать его после внесения изменений.

2. Для анализа сообщений ядра и контроля корректности работы драйвера использовалась утилита:

- `dmesg` – позволяет просматривать журнал ядра Linux. С её помощью отслеживались сообщения о регистрации драйвера, подключении устройства, а также возможные ошибки или предупреждения в процессе обработки HID-событий.

3. Для проверки работы подсистемы ввода и анализа событий мыши использовалась утилита:

- `evtest` – инструмент пользовательского пространства, предназначенный для чтения и отображения событий из устройств `/dev/input/eventX`. Это консольная

программа, позволяющая считывать и отображать события ввода.

Использовалась для подтверждения корректности определения кодов кнопок (BTN\_EXTRA, BTN\_SIDE) и формирования событий эмуляции клавиш.

4. Для идентификации подключённого HID-устройства и проверки его USB-идентификаторов применялась команда:

- lsusb – позволяет получить список USB-устройств, подключённых к системе.

С её помощью был определён идентификатор мыши Logitech G502 (046d:c08b), использованный при регистрации HID-драйвера.

5. Для настройки макросов использовался стандартный интерфейс sysfs через запись в файлы /sys/bus/hid/drivers/hid\_mouse\_macro/\*/macro\_btnN.

Применение данных инструментов обеспечило полный цикл разработки драйвера: от загрузки и диагностики до тестирования работы устройства ввода на уровне событий.

### **3.2. Структура программного проекта**

Структура программного проекта

driver

- hid\_mouse\_macro.c
- Makefile

1 directory, 2 files

Представлен каталог проекта после сборки модуля. Основной исходный файл hid\_mouse\_macro.c содержит реализацию всей логики драйвера: регистрацию HID-драйвера в ядре, обработку входных HID-событий, создание виртуального устройства ввода для эмуляции клавиш, поддержку sysfs-интерфейса и управление отложенным выполнением макросов. Для сборки модуля используется стандартный KBuild-совместимый Makefile, который обеспечивает компиляцию исходного кода с учётом конфигурации и версии текущего ядра Linux. Makefile определяет имя конечного модуля, путь к дереву исходников ядра и набор правил для сборки, очистки, загрузки и выгрузки модуля.

### **3.3. Общий алгоритм работы драйвера**

Драйвер функционирует по следующему алгоритму:

1. При загрузке модуля (module\_init) регистрируется в HID-подсистеме ядра с указанием поддерживаемых устройств (id\_table).
2. При обнаружении совместимого HID-устройства драйвер автоматически захватывает его.
3. Все входящие HID-события от устройства перехватываются в обработчике event.

4. Для событий типа EV\_KEY с кодами BTN\_EXTRA, BTN\_SIDE и другими программируемыми кнопками проверяется наличие назначенного макроса.
5. Если макрос назначен, запускается отложенная работа (delayed\_work) с задержкой 500 мс.
6. По истечении задержки выполняется эмуляция нажатия клавиш через виртуальное устройство ввода (input\_dev), созданное при инициализации.
7. Макросы настраиваются пользователем через запись в sysfs-файлы вида macro\_btn0, macro\_btn1 и т.д.
8. При выгрузке модуля все ресурсы корректно освобождаются.

### **3.4. Реализация ключевых функций драйвера**

#### Заголовочные файлы

В начале исходного файла драйвера подключаются заголовочные файлы ядра Linux <linux/module.h>, <linux/hid.h>, <linux/input.h>, <linux/delay.h>, <linux/workqueue.h> и другие.

Заголовочный файл <linux/module.h> содержит определения и макросы, необходимые для создания загружаемых модулей ядра Linux. В частности, он предоставляет интерфейсы для:

1. регистрации и удаления модуля ядра;
2. описания метаданных модуля (лицензия, автор, версия, описание);
3. использования макросов module\_init и module\_exit, определяющих функции инициализации и завершения работы драйвера.

Заголовочный файл <linux/hid.h> содержит определения структур и функций, относящихся к подсистеме Human Interface Device (HID). Он используется для реализации драйверов устройств класса HID и предоставляет:

1. структуру hid\_driver, описывающую HID-драйвер;
2. структуру hid\_device, представляющую HID-устройство;
3. интерфейсы для регистрации драйвера в HID-подсистеме ядра Linux.

Заголовочные файлы <linux/input.h> и <linux/workqueue.h> необходимы для создания виртуального устройства ввода и управления отложенным выполнением.

#### Параметры модуля

В отличие от драйвера зеркалирования, в данном драйвере параметры модуля не используются, так как настройка макросов осуществляется динамически через sysfs-интерфейс. Это обеспечивает гибкость: пользователь может изменить макрос в любое время без перезагрузки модуля.

#### Обработка событий мыши через обработчик event

Центральной функцией драйвера является обработчик `macro_event`, предназначенный для перехвата событий HID-устройств после их парсинга HID-подсистемой. Данная функция вызывается для каждого события типа `EV_KEY`. В начале функции выполняется проверка: если тип события не `EV_KEY` или значение не равно 1 (нажатие), событие игнорируется.

Далее код кнопки (`usage->code`) сопоставляется с известными кодами дополнительных кнопок (`BTN_EXTRA = 276`, `BTN_SIDE = 275` и др.) с помощью функции `btn_code_to_index`. Если код найден и для соответствующего индекса назначен макрос, устанавливается флаг `pending` и запускается отложенная работа:

```
schedule_delayed_work(&mdev->macro_work, msecs_to_jiffies(500));
```

Функция возвращает 1, что указывает HID-подсистеме подавить исходное событие (оно не будет передано в `evdev`). Это предотвращает дублирование: пользователь не получает события от физической кнопки, только результат макроса.

#### Таблица идентификаторов устройств

Для обеспечения привязки драйвера к целевым устройствам объявлена статическая константная таблица `mouse_macro_table` типа `hid_device_id`. Эта таблица содержит элементы, инициализированные макросом `HID_USB_DEVICE`, включающим идентификатор производителя (Vendor ID) и идентификатор продукта (Product ID) конкретного HID-устройства:

```
с
static const struct hid_device_id mouse_macro_table[] = {
    { HID_USB_DEVICE(0x046d, 0xc08b) }, // Logitech G502
    { HID_USB_DEVICE(0x03f0, 0x1c41) }, // HP Mouse
    { } // терминатор
};
```

Обязательным требованием HID-подсистемы является завершение таблицы пустой записью `{ }`, играющей роль терминатора списка. Без неё механизм сопоставления устройств может привести к неопределённому поведению. Для обеспечения автоматического распознавания драйвера системой управления модулями (`modprobe`, `udev`) используется макрос `MODULE_DEVICE_TABLE(hid, mouse_macro_table)`.

## Функция инициализации

Функция `macro_probe` вызывается при обнаружении совместимого HID-устройства. В ней:

1. выделяется память под структуру устройства;
2. инициализируется мьютекс и `workqueue`;
3. создаётся и регистрируется виртуальное устройство ввода (`input_dev`) для эмуляции клавиш;
4. создаются `sysfs`-атрибуты (`macro_btn0 ... macro_btn7`);
5. запускается HID-устройство через `hid_hw_start`.

Виртуальное устройство ввода настраивается как клавиатура: устанавливаются биты `EV_KEY` и все возможные клавиши (`KEY_MAX`), чтобы поддерживать любые макросы.

## Функция завершения работы

Функция `macro_remove` вызывается при отключении устройства или выгрузке модуля. В ней:

1. отменяется отложенная работа (`cancel_delayed_work_sync`);
2. останавливается HID-устройство (`hid_hw_stop`);
3. удаляются `sysfs`-атрибуты;
4. выгружается виртуальное устройство ввода;
5. освобождается память.

Это гарантирует корректное завершение работы и отсутствие утечек ресурсов.

## Глава 4. Тестирование

### *Шаг 1. Идентификация устройства в системе*

Выполнена команда:

`lsusb`

Результат:

Bus 003 Device 007: ID 03f0:1c41 HP, Inc HP Gaming Mouse

```
tabsent@DEBIAN:~/driver$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 003 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 002: ID 0bc2:2030 Seagate RSS LLC Expansion HDD
Bus 003 Device 003: ID 14cd:8608 Super Top Hub [Super Top]
Bus 003 Device 004: ID 5986:2127 Bison Electronics Inc. HD Camera
Bus 003 Device 005: ID 214b:7250 Huasheng Electronics USB2.0 HUB
Bus 003 Device 006: ID 06cb:009b Synaptics, Inc.
Bus 003 Device 007: ID 03f0:1c41 HP, Inc HP Gaming Mouse
Bus 003 Device 010: ID 8087:0026 Intel Corp. AX201 Bluetooth
Bus 003 Device 012: ID aaaa:8816 MXT microSD CardReader
Bus 004 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
```

Это подтверждает, что система корректно распознала устройство на уровне USB.

### *Шаг 2. Проверка текущего драйвера*

Выполнена команда:

`ls /sys/bus/hid/drivers/`

До загрузки пользовательского драйвера мышь управляется стандартным драйвером `hid-generic` и `hid-multitouch`.

```
tabsent@DEBIAN:~/driver$ ls /sys/bus/hid/drivers/
hid-generic  hid-multitouch
```

### Шаг 3. Сборка и загрузка драйвера

Выполнены команды:

make

sudo make load

```
tabsent@DEBIAN:~/driver$ make
make -C /lib/modules/6.12.57+deb13-amd64/build M=/home/tabsent/driver modules
make[1]: вход в каталог «/usr/src/linux-headers-6.12.57+deb13-amd64»
  CC [M]  /home/tabsent/driver/hid_mouse_macro.o
  MODPOST /home/tabsent/driver/Module.symvers
  CC [M]  /home/tabsent/driver/hid_mouse_macro.mod.o
  CC [M]  /home/tabsent/driver/.module-common.o
  LD [M]  /home/tabsent/driver/hid_mouse_macro.ko
  BTF [M] /home/tabsent/driver/hid_mouse_macro.ko
make[1]: выход из каталога «/usr/src/linux-headers-6.12.57+deb13-amd64»
tabsent@DEBIAN:~/driver$ sudo make load
sudo insmod hid_mouse_macro.ko
```

Вывод dmesg:

[mouse\_macro] Attached to HP HP Gaming Mouse (VID:0x03f0 PID:0x1c41)

```
[ 2358.861703] hid_mouse_macro: loading out-of-tree module taints kernel.
[ 2358.861716] hid_mouse_macro: module verification failed: signature and/or required key
missing - tainting kernel
[ 2358.931520] input: mouse_macro_emu as /devices/pci0000:00/0000:00:14.0/usb3/3-3/3-3.3/3-3.3:1.0/0003:03F0:1C41.0002/input/input24
[ 2358.931942] input: HP HP Gaming Mouse as /devices/pci0000:00/0000:00:14.0/usb3/3-3/3-3.3:1.0/0003:03F0:1C41.0002/input/input25
[ 2358.932547] hid_mouse_macro 0003:03F0:1C41.0002: input,hidraw1: USB HID v1.11 Mouse [HP HP Gaming Mouse] on usb-0000:00:14.0-3.3/input0
[ 2358.932558] [mouse_macro] Attached to HP HP Gaming Mouse (VID:0x03f0 PID:0x1c41)
[ 2358.933661] input: mouse_macro_emu as /devices/pci0000:00/0000:00:14.0/usb3/3-3/3-3.3:1.1/0003:03F0:1C41.0003/input/input26
[ 2358.934365] hid_mouse_macro 0003:03F0:1C41.0003: hiddev0,hidraw2: USB HID v1.11 Device [HP HP Gaming Mouse] on usb-0000:00:14.0-3.3/input1
[ 2358.934375] [mouse_macro] Attached to HP HP Gaming Mouse (VID:0x03f0 PID:0x1c41)
```

Это подтверждает успешную загрузку и привязку драйвера к устройству.

### Шаг 4. Назначение макросов

Выполнены команды:

echo "29,46" | sudo tee /sys/bus/hid/drivers/hid\_mouse\_macro/\*/macro\_btn1 #

BTN\_SIDE → Ctrl+C

echo "29,47" | sudo tee /sys/bus/hid/drivers/hid\_mouse\_macro/\*/macro\_btn0 #

BTN\_EXTRA → Ctrl+V

```
tabsent@DEBIAN:~/driver$ echo "29,46" | sudo tee /sys/bus/hid/drivers/hid_mouse_macro/*/macro_btn1
29,46
tabsent@DEBIAN:~/driver$ echo "29,47" | sudo tee /sys/bus/hid/drivers/hid_mouse_macro/*/macro_btn0
29,47
```

Вывод dmesg:

```
[ 2541.899519] [mouse_macro] Macro for btn1 set (2 keys)
[ 2541.899529] [mouse_macro] Macro for btn1 set (2 keys)
[ 2553.174652] [mouse_macro] Macro for btn0 set (2 keys)
[ 2553.174660] [mouse_macro] Macro for btn0 set (2 keys)
```

### ***Шаг 5. Визуальная проверка в графической среде***

В текстовом редакторе gedit:

1. Выделен текст.
2. Нажата боковая кнопка (BTN\_SIDE) → текст скопирован.
3. Нажата вторая боковая кнопка (BTN\_EXTRA) → текст вставлен.

```
[ 2769.270829] [mouse_macro] Button BTN_SIDE (code 275) pressed -> executing macro after 500ms:
[ 2769.270838] [mouse_macro] -> press KEY_29
[ 2769.270841] [mouse_macro] -> press KEY_46
[ 2769.280787] [mouse_macro] -> release KEY_46
[ 2769.280792] [mouse_macro] -> release KEY_29
[ 2818.966848] [mouse_macro] Button BTN_EXTRA (code 276) pressed -> executing macro after 500ms:
[ 2818.966858] [mouse_macro] -> press KEY_29
[ 2818.966861] [mouse_macro] -> press KEY_47
[ 2818.976808] [mouse_macro] -> release KEY_47
[ 2818.976811] [mouse_macro] -> release KEY_29
```

Поведение соответствует ожидаемому. Задержка 0.5 секунды также подтверждена визуально.

### ***Шаг 6. Проверка устойчивости и отмены***

Выполнена команда:

```
sudo make unload
```

Вывод dmesg:

```
[ 2938.342974] [mouse_macro] Detached from HP HP Gaming Mouse
[ 2938.491185] [mouse_macro] Detached from HP HP Gaming Mouse
[ 2938.491504] input: HP HP Gaming Mouse as /devices/pci0000:00/0000:00:14.0/usb3/3-3/3-3.3/3-3.3:1
.0/0003:03F0:1C41.0002/input/input27
[ 2938.491957] hid-generic 0003:03F0:1C41.0002: input,hidraw1: USB HID v1.11 Mouse [HP HP Gaming Mo
use] on usb-0000:00:14.0-3.3/input0
[ 2938.492656] hid-generic 0003:03F0:1C41.0003: hiddev0,hidraw2: USB HID v1.11 Device [HP HP Gaming
Mouse] on usb-0000:00:14.0-3.3/input1
```

Мышь возвращается под управление стандартного драйвера. Система стабильна, ошибок ядра не возникает.

## Вывод

В ходе выполнения курсовой работы была успешно реализована поставленная цель – разработан и протестирован драйвер операционной системы Linux, обеспечивающий программирование макросов на дополнительных кнопках компьютерной мыши на уровне ядра. Все запланированные задачи выполнены в полном объёме: изучены архитектура подсистемы ввода и принципы работы HID-устройств, проанализированы механизмы обработки событий мыши, спроектирована и реализована рабочая версия драйвера с поддержкой sysfs-интерфейса, корректной эмуляции сочетаний клавиш и задержки выполнения, а также проведено комплексное тестирование с использованием системных инструментов.

В процессе работы получены практические навыки системного программирования на языке C в контексте ядра Linux, включая работу с HID-событиями, управление виртуальными устройствами ввода, обработку отложенных работ и отладку драйверов. Особое внимание уделено реальным инженерным аспектам, таким как конфликт драйверов, необходимость корректного управления ресурсами и валидация поведения через инструментальные и визуальные методы.

Разработанное решение корректно осуществляет обработку событий от мыши и позволяет гибко настраивать поведение кнопок через sysfs, что подтверждено тестированием. Таким образом, курсовая работа не только закрепляет теоретические знания, но и формирует ценностный практический опыт, необходимый для дальнейшей деятельности в области системного программирования, разработки драйверов и анализа низкоуровневых компонентов ОС.

---

# Приложения

## Приложение 1. Исходный код драйвера (*hid\_mouse\_macro.c*)

```
// hid_mouse_macro.c
// HID драйвер для программирования макросов на кнопках мыши (Debian 13 /
Linux 6.12+)
// Режим ядра (kernel mode), без DPI, только макросы

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/hid.h>
#include <linux/input.h>
#include <linux/slab.h>
#include <linux/device.h>
#include <linux/mutex.h>
#include <linux/delay.h>
#include <linux/string.h>

#define DRIVER_NAME "hid_mouse_macro"
#define MAX_MACRO_KEYS 10

// Структура одного макроса
struct macro_entry {
    bool enabled;
    u16 keys[MAX_MACRO_KEYS]; // массив кодов клавиш (KEY_A = 30 и т.д.)
    int count;
};

// Контекст устройства
struct mouse_macro_device {
    struct hid_device *hdev;
    struct input_dev *macro_input; // виртуальное устройство для эмуляции
    struct macro_entry macros[8]; // макросы для 8 программируемых кнопок
    struct mutex lock;
};

// Сопоставление HID-кодов кнопок с индексами макросов
static int btn_code_to_index(unsigned int code)
{
    switch (code) {
        case BTN_EXTRA: return 0; // макрос macro_btn0
        case BTN_SIDE: return 1; // macro_btn1
        case BTN_FORWARD: return 2; // macro_btn2
```

```

    case BTN_BACK:    return 3; // macro_btn3
    case BTN_TASK:    return 4; // macro_btn4
    case BTN_TRIGGER: return 5; // macro_btn5
    case BTN_THUMB:   return 6; // macro_btn6
    case BTN_THUMB2:  return 7; // macro_btn7
    default:          return -1;
}
}

// Выполнение макроса с поддержкой удержания (Ctrl+C)
static void execute_macro(struct input_dev *emu_dev, struct macro_entry *m,
unsigned int btn_code)
{
    const char *btn_name;
    int i;

    switch (btn_code) {
        case BTN_EXTRA:  btn_name = "BTN_EXTRA"; break;
        case BTN_SIDE:   btn_name = "BTN_SIDE"; break;
        case BTN_FORWARD: btn_name = "BTN_FORWARD"; break;
        case BTN_BACK:   btn_name = "BTN_BACK"; break;
        default:          btn_name = "UNKNOWN";
    }

    printk(KERN_INFO "[mouse_macro] Button %s (code %u) pressed -> executing
macro (hold mode)\n",
        btn_name, btn_code);

    // Нажать все клавиши одновременно
    for (i = 0; i < m->count; i++) {
        printk(KERN_INFO "[mouse_macro] -> press KEY_%u\n", m->keys[i]);
        input_report_key(emu_dev, m->keys[i], 1);
    }
    input_sync(emu_dev);
    mdelay(10); // небольшая задержка

    // Отпустить все клавиши
    for (i = 0; i < m->count; i++) {
        printk(KERN_INFO "[mouse_macro] -> release KEY_%u\n", m->keys[i]);
        input_report_key(emu_dev, m->keys[i], 0);
    }
    input_sync(emu_dev);
}

// Обработчик HID-событий (уже обработанных ядром)

```

```

static int macro_event(struct hid_device *hdev, struct hid_field *field,
                      struct hid_usage *usage, __s32 value)
{
    struct mouse_macro_device *mdev = hid_get_drvdata(hdev);
    unsigned int code = usage->code;
    int idx;

    // Интересуют только нажатия кнопок (value == 1)
    if (usage->type != EV_KEY || value != 1)
        return 0;

    idx = btn_code_to_index(code);
    if (idx < 0)
        return 0; // не поддерживаемая кнопка

    mutex_lock(&mdev->lock);
    if (idx < 8 && mdev->macros[idx].enabled) {
        execute_macro(mdev->macro_input, &mdev->macros[idx], code);
        mutex_unlock(&mdev->lock);
        return 1; // подавить исходное событие
    }
    mutex_unlock(&mdev->lock);
    return 0;
}

// Sysfs: запись макроса
static ssize_t macro_store(struct device *dev, struct device_attribute *attr,
                          const char *buf, size_t count)
{
    struct hid_device *hdev = to_hid_device(dev);
    struct mouse_macro_device *mdev = hid_get_drvdata(hdev);
    int idx;
    char *p, *s;
    u16 key;
    int i = 0;

    if (sscanf(attr->attr.name, "macro_btn%d", &idx) != 1 ||
        idx < 0 || idx >= 8)
        return -EINVAL;

    mutex_lock(&mdev->lock);
    struct macro_entry *m = &mdev->macros[idx];
    m->count = 0;
    m->enabled = false;

```

```

p = (char *)buf;
while ((s = strsep(&p, ",")) != NULL && i < MAX_MACRO_KEYS) {
    if (kstrtou16(s, 10, &key) == 0) {
        m->keys[i++] = key;
    } else {
        printk(KERN_WARNING "[mouse_macro] Invalid key code: %s\n", s);
    }
}

if (i > 0) {
    m->count = i;
    m->enabled = true;
    printk(KERN_INFO "[mouse_macro] Macro for btn%d set (%d keys)\n", idx, i);
}
mutex_unlock(&mdev->lock);
return count;
}

// Sysfs: чтение (подсказка)
static ssize_t macro_show(struct device *dev, struct device_attribute *attr, char *buf)
{
    return sprintf(buf, "Enter comma-separated key codes (e.g., 29,46 for Ctrl+C)\n");
}

// Генерация атрибутов macro_btn0 ... macro_btn7
#define MACRO_ATTR(n) static DEVICE_ATTR(macro_btn##n, 0644,
macro_show, macro_store)
MACRO_ATTR(0); MACRO_ATTR(1); MACRO_ATTR(2); MACRO_ATTR(3);
MACRO_ATTR(4); MACRO_ATTR(5); MACRO_ATTR(6); MACRO_ATTR(7);

static struct attribute *macro_attrs[] = {
    &dev_attr_macro_btn0.attr,
    &dev_attr_macro_btn1.attr,
    &dev_attr_macro_btn2.attr,
    &dev_attr_macro_btn3.attr,
    &dev_attr_macro_btn4.attr,
    &dev_attr_macro_btn5.attr,
    &dev_attr_macro_btn6.attr,
    &dev_attr_macro_btn7.attr,
    NULL
};

static const struct attribute_group macro_attr_group = {
    .attrs = macro_attrs,
};

```

```

// Таблица поддерживаемых устройств (на основе hid_mouse_dpi.txt)
static const struct hid_device_id mouse_macro_table[] = {
    { HID_USB_DEVICE(0x046d, 0xc08b) }, // Logitech G502
    { HID_USB_DEVICE(0x03f0, 0x1c41) }, // HP Mouse
    // Раскомментируйте, если нужно тестировать на любой мыши:
    // { HID_USB_DEVICE(HID_ANY_ID, HID_ANY_ID) },
    { } // терминатор — обязательно!
};
MODULE_DEVICE_TABLE(hid, mouse_macro_table);

// Инициализация устройства
static int macro_probe(struct hid_device *hdev, const struct hid_device_id *id)
{
    struct mouse_macro_device *mdev;
    int ret, i;

    mdev = kzalloc(sizeof(*mdev), GFP_KERNEL);
    if (!mdev)
        return -ENOMEM;

    mdev->hdev = hdev;
    hid_set_drvdata(hdev, mdev);
    mutex_init(&mdev->lock);

    // Создаём виртуальное устройство ввода для эмуляции макросов
    mdev->macro_input = input_allocate_device();
    if (!mdev->macro_input) {
        ret = -ENOMEM;
        goto err_free;
    }

    mdev->macro_input->name = "mouse_macro_emu";
    mdev->macro_input->id.bustype = BUS_VIRTUAL;
    mdev->macro_input->dev.parent = &hdev->dev;
    __set_bit(EV_KEY, mdev->macro_input->evbit);
    for (i = 0; i < KEY_MAX; i++)
        __set_bit(i, mdev->macro_input->keybit);

    ret = input_register_device(mdev->macro_input);
    if (ret)
        goto err_input;

    ret = sysfs_create_group(&hdev->dev.kobj, &macro_attr_group);
    if (ret)

```

```

    goto err_sysfs;

ret = hid_parse(hdev);
if (ret)
    goto err_parse;

ret = hid_hw_start(hdev, HID_CONNECT_DEFAULT);
if (ret)
    goto err_start;

printk(KERN_INFO "[mouse_macro] Attached to %s (VID:0x%04x PID:0x%04x)
\n",
        hdev->name, hdev->vendor, hdev->product);
return 0;

err_start:
    sysfs_remove_group(&hdev->dev.kobj, &macro_attr_group);
err_parse:
    input_unregister_device(mdev->macro_input);
err_sysfs:
err_input:
    kfree(mdev);
err_free:
    return ret;
}

// Удаление устройства
static void macro_remove(struct hid_device *hdev)
{
    struct mouse_macro_device *mdev = hid_get_drvdata(hdev);
    hid_hw_stop(hdev);
    sysfs_remove_group(&hdev->dev.kobj, &macro_attr_group);
    input_unregister_device(mdev->macro_input);
    kfree(mdev);
    printk(KERN_INFO "[mouse_macro] Detached from %s\n", hdev->name);
}

// Регистрация HID-драйвера
static struct hid_driver macro_hid_driver = {
    .name = DRIVER_NAME,
    .id_table = mouse_macro_table,
    .probe = macro_probe,
    .remove = macro_remove,
    .event = macro_event,    // используем event, не raw_event
};

```

```
module_hid_driver(macro_hid_driver);
```

```
MODULE_LICENSE("GPL");
```

```
MODULE_AUTHOR("Student");
```

```
MODULE_DESCRIPTION("HID Mouse Button Macro Driver for Debian 13 (Linux  
6.12+)");
```

```
MODULE_VERSION("1.0");
```

## ***Приложение 2. Makefile***

```
makefile
```

```
obj-m += hid_mouse_macro.o
```

```
KERNEL_DIR ?= /lib/modules/$(shell uname -r)/build
```

```
PWD := $(shell pwd)
```

```
all:
```

```
$(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules
```

```
clean:
```

```
$(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean
```

```
load:
```

```
sudo insmod hid_mouse_macro.ko
```

```
unload:
```

```
sudo rmmod hid_mouse_macro
```